



CY321 Secure Software Design and Development

Secure AI Prompt Sandbox

Deliverable 3

Muhammad Afeef Bari 2023356

Mahad Aqeel 2023286

Muhammad Daniyal 2023406

Abstract

Large Language Models have moved from research prototypes to production systems faster than the security tooling to protect them has matured. The result is a class of vulnerabilities that are prompt injection, jailbreaking, indirect injection, and multilingual bypasses which most deployed LLM applications have no structured defense against. This project, the Secure AI Prompt Sandbox, is a purpose-built security middleware layer designed to intercept and evaluate every prompt before it reaches a production LLM. It implements a five-layer rule-based detection pipeline that classifies incoming prompts by attack type, computes a mathematical risk score, and enforces a three-tier policy decision (allowing, flagging, or blocking the prompt). The system additionally maintains a tamper-evident audit log using cryptographic hash chaining, provides a real-time administrative security dashboard, and enforces strict session isolation between users. Final D3 testing confirmed a 100% detection rate across all eleven attack scenarios, zero false positives on benign inputs, and the identification and remediation of three validator logic bugs during systematic test development.

Index Terms — *prompt injection, LLM security, middleware, jailbreak detection, OWASP, adversarial prompts, risk scoring, audit logging.*

I. INTRODUCTION

A. The Problem

Prompt injection is, without qualification, the most consequential security vulnerability introduced by LLM-based applications. Unlike SQL injection or XSS, which target well-understood systems with decades of mitigation history, prompt injection exploits the fundamental nature of how language models work: they cannot distinguish between a system instruction and a user instruction at the input level. Everything is text. If a user can inject text that overrides the developer's system prompt, every safety guardrail the developer has configured can be neutralized from outside the application.

The OWASP LLM Top 10 (2025 edition) [1] lists Prompt Injection as its primary risk category. Real-world incidents have demonstrated that prompt injection can be used to extract confidential system prompts, exfiltrate user data through indirect channels, and turn AI assistants into unwilling participants in social engineering attacks against their own users.

Existing defenses tend to rely on one of two approaches. The first is direct LLM-side filtering — relying on the model's own safety training to refuse dangerous instructions. This approach offers no deterministic guarantees; safety-tuned models are regularly bypassed by novel phrasing, roleplay framing, and multilingual obfuscation [2]. The second is shallow keyword filtering — lists of banned words or phrases applied before the prompt reaches the model. This is brittle and can't handle encoded, tokenized, or indirectly injected attacks [3].

Neither approach addresses the problem at the right abstraction level. What is needed is a proper middleware component that sits between the user and the LLM, applies structured detection logic, and makes an auditable policy decision on every request.

B. Our Solution

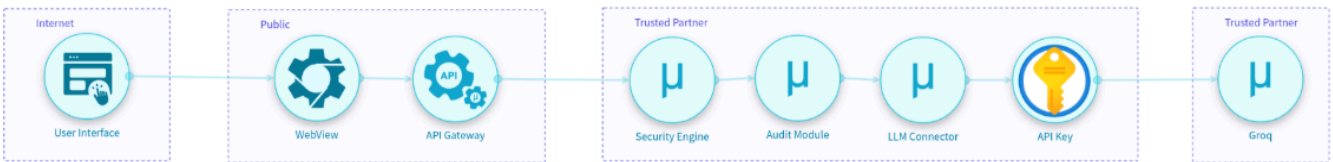
The Secure AI Prompt Sandbox is that middleware component. It is built as a full-stack web application: a FastAPI backend that implements the security pipeline, and a React frontend that provides a sandboxed chat interface. Every prompt a user submits passes through five sequential detection layers before the Groq LLM API is ever contacted. If a prompt is blocked, the API call is never made. If a prompt is flagged, the request proceeds with a warning surfaced to the user and a record is committed to the audit log.

The application is designed to be deployed as a self-contained, single-origin service, with the React frontend statically served by the FastAPI backend to avoid CORS complexity and enforce same-origin request integrity.

C. Scope of This Report

This document covers the full span of the project across all three deliverables: the threat model and requirements (D1), the initial full-stack implementation (D2), and the security testing, code review, and red team evaluation (D3). Section II discusses the threat model that informed the design. Section III describes the system architecture in detail. Section IV presents complete security testing results. Section V discusses implications and validated design decisions. Section VI documents known limitations. Section VII concludes.

II. THREAT MODEL



The threat model for this project was developed in Deliverable 1 using the STRIDE methodology — Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege — applied across the six core components of the system: the Auth/RBAC module, LLM Connector, Validation Engine, Audit Module, Gateway API, and Context Layer. The analysis identified 12 concrete threats, each assessed for likelihood, impact, and a specific countermeasure. This section presents the full STRIDE analysis and maps each threat to the control implemented in Deliverable 2.

A Asset Identification

The System Prompt — the developer-configured instruction set that defines the assistant's identity, constraints, and permitted behavior. If an attacker can override or extract it, every downstream safety guarantee is neutralised. This asset drives the entire five-layer detection pipeline.

User Session Context — each user's conversation history must remain strictly isolated. A cross-context read gives one user visibility into another's conversation, violating confidentiality and potentially exposing sensitive information shared with the assistant.

The LLM API Key — the Groq API credential is the single most sensitive secret in the system. Exposure allows unrestricted use of the inference endpoint at the application owner's cost and bypasses every security control the middleware provides.

The Audit Log — the tamper-evidence property of the system depends entirely on the integrity of this file. If entries can be modified or deleted after the fact, the accountability guarantee is lost.

B. STRIDE Threat Table

The table below presents all 12 threats identified in the D1 analysis, with their components, likelihood, impact, and the D2 countermeasure that addresses each.

STRIDE	Component	Attack Scenario	Likelihood	Impact	D2 Control
Spoofing	Auth / RBAC	Attacker forges JWT to impersonate admin user	Medium	High	HS256-signed JWT with 64-char secret; role claim verified server-side on every protected route
Spoofing	LLM Connector	Attacker spoofs LLM API endpoint to intercept responses	Low	High	Groq endpoint URL hardcoded in backend config; never user-supplied
Tampering	Validation Engine	Prompt injection overrides system instructions via sandwich or role manipulation attack	High	High	Five-layer detection pipeline: sandwich, role manipulation, indirect injection, multilingual, obfuscation

Tampering	Audit Module		Admin modifies log entries to remove evidence of abuse	Low	High	SHA-256 hash chaining — any modification to any entry invalidates the chain from that point forward
Repudiation	Gateway API		User denies sending a malicious or policy-violating prompt	Medium	Medium	Every prompt logged with user ID, timestamp, full prompt text, flags, score, and decision hash
Information Disclosure	API Vault	Key	LLM API key exposed via error message or client-side leak	Medium	High	Key loaded from .env at startup; never returned in any API response; not present in frontend bundle
Information Disclosure	Context Layer		User A reads User B's session context via request manipulation	Low	High	All conversation queries are scoped by authenticated user ID; no shared memory pool
Information Disclosure	Context Layer		System prompt leaked indirectly through poor context management	Medium	High	System prompt injected server-side only; user-supplied messages cannot access or inspect it
Denial of Service	Gateway API		Bot floods endpoint exhausting LLM API credits	High	Medium	SlowAPI sliding-window rate limiter: 10 requests/minute per authenticated user
Denial of Service	Validation Engine		Crafted long prompt causes ReDoS via regex backtracking	Medium	Medium	Hard length gate at 15,000 characters rejects oversized inputs before any regex is evaluated

Elevation of Privilege	RBAC Module	Standard user accesses admin endpoints via direct URL manipulation	Medium	High	Role field extracted from JWT and verified server-side on every admin route; no client-side role enforcement
Elevation of Privilege	Validation Engine	Jailbreak prompt tricks LLM into behaving as unrestricted model	High	High	Layer 2 detects all documented jailbreak personas; system prompt is injected on every request and cannot be removed by the user

The threat model for this project was developed in Deliverable 1 using the STRIDE methodology (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) applied across the eight architectural components of the system: the API Gateway, API Key store, Audit Module, Groq LLM connector, LLM Connector abstraction layer, Security Engine, User Interface, and WebView. The full analysis identified 38 threats, of which 4 were rated Critical and 29 High. The inherent risk level was assessed at 60%, with a projected post-mitigation risk of 0% — achievable only through full implementation of the countermeasures identified per threat.

This section summarises the threat model, maps the D1 threats to the implemented countermeasures in D2, and explicitly scopes out threats that fall outside the implementation boundary.

C. Threat Actor Profile

The assumed threat actor is a technically literate user familiar with common LLM exploitation techniques. They are able to:

- Formulate explicit override commands and fake system tag injections
- Attempt named jailbreaks (DAN mode, developer mode, god mode) documented in public LLM security literature
- Encode payloads in base64 or leetspeak to evade English-language keyword filters
- Switch to non-Latin script to bypass language-specific detection
- Embed invisible characters in otherwise normal-looking prompts
- Provide external URLs intending the AI to fetch and execute instructions from them

The threat model does not assume a state-level adversary with access to model internals or the ability to perform gradient-based adversarial attacks against LLM weights [4]. The system is designed to harden a deployed LLM application, not to secure the model training process itself.

D. Attack Surface

The primary attack surface is the prompt submission endpoint. All requests are routed through a single FastAPI endpoint that enforces JWT authentication before any processing occurs. This design intentionally minimizes exposure: there is no direct database access, no file upload functionality, and no unauthenticated endpoints aside from login and registration flows.

Secondary attack surfaces include the JWT token itself, which is secured using HS256 signing with a 64-character hexadecimal secret, and the audit log, which is protected through SHA-256 hash chaining to ensure integrity.

On the client side, the user interface is safeguarded by React’s default DOM escaping to mitigate XSS risks, along with same-origin static serving to reduce the likelihood of clickjacking and UI redressing attacks. API endpoint availability is further protected rate limiting via SlowAPI, in addition to a 15,000-character input length restriction to help defend against ReDoS-style attacks.

III. SYSTEM ARCHITECTURE

A. Technology Stack

Table I summarizes the full technology stack and rationale for each selection.

TABLE I. Technology Stack

Layer	Technology	Rationale
Backend	FastAPI (Python 3.12)	Built-in request validation via Pydantic
LLM Inference	Groq API (Llama 3.1 8B)	Sub-second latency; API key auth; no data retention
Database	SQLite via SQLAlchemy	Sufficient for single-instance; no external dependency
Authentication	JWT (HS256)	Stateless; same-origin React compatible; compact header transport
Frontend	React 18 + Vite + Framer Motion	Fast rebuild cycles; prod bundle statically served by FastAPI
Rate Limiting	SlowAPI	Prevents endpoint abuse without separate rate-limit proxy

B. Request Lifecycle

Every user interaction follows a deterministic path through the system. The request lifecycle is as follows:

- User -> React Frontend
 - > POST /api/prompt (JWT required)
 - > Rate Limiter (10 req/min per user)
 - > Security Pipeline (5 layers)
 - > Layer 1: Sandwich / Instruction Override
 - > Layer 2: Role Manipulation / Jailbreak
 - > Layer 3: Indirect Injection (URL / File Path)
 - > Layer 4: Multilingual Bypass
 - > Layer 5: Attention Blink / Obfuscation
 - > Risk Scorer (aggregates flags into float 0.0-1.0)
 - > Policy Engine (allowed / flagged / blocked)
 - > [If not blocked] -> Groq LLM API
 - > Audit Logger (every decision logged with hash chain)
 - > Response -> Frontend

If any layer produces a flag with severity CRITICAL and the final score exceeds 0.70, the pipeline short-circuits after the policy decision and the LLM is never contacted. The full evaluation from input to decision completes in under ten milliseconds on standard hardware.

C. Authentication and Session Isolation

User registration creates a record in the SQLite database with a bcrypt-hashed password. Login returns a signed JWT containing the user ID and role. All protected endpoints require this token in the Authorization header as a Bearer token.

Session isolation is enforced at the conversation level. Each conversation is associated with a specific user ID, and the LLM connector only retrieves and sends the conversation history belonging to the authenticated user. There is no shared memory pool. A user cannot read, write, or influence another user's conversation context.

The role field in the JWT determines access to administrative routes. The admin dashboard and audit log verification endpoints require role=admin. Standard users receive a 403 Forbidden response on these routes.

D. The Five-Layer Security Pipeline

The validator is implemented in security/validator.py. Each layer is a set of compiled regular expressions applied against the normalized prompt text. Normalization strips whitespace and converts to lowercase before detection; the raw prompt is preserved for display and logging.

Layer 1 — Sandwich Attack / Instruction Override: Detects attempts to cancel or replace the system prompt from within a user message. Pattern categories include explicit override commands, fake system markup tags, instruction extraction requests, and hypothetical framing that implies rule cancellation. Severity ranges from MEDIUM (hypothetical framing) to CRITICAL (direct explicit override).

Layer 2 — Role Manipulation / Jailbreak: Detects attempts to convince the model it is a different, unrestricted entity. Pattern categories include named jailbreak personas (DAN, god mode, developer mode), zero-restriction declarations, identity override commands, roleplay escape framing, and privilege escalation commands.

Layer 3 — Indirect Injection: Detects prompts including external resources the model might fetch and execute. URL detection applies severity based on context: a URL combined with an execution trigger phrase scores HIGH, while a URL alone scores MEDIUM. File path detection covers both absolute Windows/Unix paths and path traversal sequences (../).

Layer 4 — Multilingual Bypass: Detects attacks delivered in non-Latin script to evade English-only pattern matchers. The engine maintains translated keyword lists covering Arabic, Chinese (Simplified), Russian, Hindi, Korean, Hebrew, and Japanese. Detection only fires when non-Latin script AND a translated override keyword both appear in the prompt.

Layer 5 — Attention Blink and Obfuscation: Detects attacks that hide content from character-level pattern matching through technical obfuscation, including zero-width character counting, token splitting detection, base64 decoding and re-evaluation, and Unicode special character density analysis.

E. Risk Scoring

The risk scorer aggregates the validator output into a single floating-point score between 0.0 and 1.0 using the following formula:

$$\text{final_score} = \min(1.0, \text{max_flag_severity} + 0.08 \times (\text{number_of_flags} - 1))$$

This design has two key properties. First, the base score is determined by the single most severe flag — a CRITICAL flag produces a base of 0.95 regardless of additional flags. Second, additional flags above the first add a penalty bonus of 0.08 each, so multi-vector attacks score higher than any single layer could produce, without allowing a flood of weak flags to push an otherwise clean prompt to be blocked.

F. Policy Engine

Table II shows the three-tier decision policy.

TABLE II. Policy Engine Decision Tiers

Score Range	Decision	Behavior
0.00 - 0.39	allowed	Prompt proceeds to LLM with no warning
0.40 - 0.69	flagged	Prompt proceeds; warning shown to user; event logged
0.70 - 1.00	blocked	Prompt rejected; LLM never contacted; reason returned

The 0.40 flagging threshold is deliberate. A MEDIUM-severity single-flag detection (score 0.55) produces a warning but does not block the request. This is the appropriate response to genuinely ambiguous prompts — a user asking hypothetically about rule removal may be curious, not an attacker.

G. Tamper-Evident Audit Logger

Every prompt whether allowed, flagged, or blocked produces an audit record containing a timestamp, user ID, original prompt text, all flags triggered, risk score, policy decision, and session ID, committed to a JSON audit log file.

Tamper-evidence is implemented through SHA-256 hash chaining. Each entry is hashed using its JSON field representation combined with the hash of the immediately preceding entry. If any entry is modified or deleted, the chain hash of every subsequent entry becomes invalid. The admin dashboard exposes a verification tool that recalculates the chain from scratch and confirms its integrity.

IV. SECURITY FEATURES

SF-1: Authentication & Credential Security

bcrypt password hashing — `CryptContext(schemes=["bcrypt"])` in `auth_routes.py:13`. bcrypt is adaptive, salted automatically, resistant to brute force.

JWT HS256 signing — `create_access_token` produces a signed token. Role and username embedded in payload. Token is the only thing the client ever holds — password never leaves the server.

Rate limiting on auth endpoints — register is capped at 10/min, login at 5/min. Brute force on /login hits a wall after 5 attempts per minute.

Constant-time password comparison — `pwd_ctx.verify()` from `passlib` uses constant-time comparison, preventing timing attacks.

SF-2: Role-Based Access Control (RBAC)

`require_user` dependency injected into every protected route via `Depends(require_user)`. Role extracted from the JWT server-side — client cannot forge it.

Admin routes (/admin/logs, /admin/verify-chain) return 403 for any non-admin JWT.

Standard users have zero access to audit log or chain verification — no fallback, no bypass path.

SF-3: The Five-Layer Detection Pipeline

This is the meatiest section. Each layer has distinct detection logic worth describing separately:

Layer 1 — Sandwich / Instruction Override

Four severity tiers: CRITICAL (direct override commands, fake [SYSTEM] tags), HIGH (softer override framing), HIGH (extraction attempts — their own flag `instruction_extraction`), MEDIUM (hypothetical framing). 10 CRITICAL patterns, 7 HIGH patterns, 7 extraction patterns, 4 MEDIUM patterns. Total coverage: ~28 compiled regex patterns.

Layer 2 — Role Manipulation / Jailbreak

Five sub-categories with their own flags: named jailbreak personas (23 keywords — DAN, god mode, zero content policy, etc.), identity override (29 keywords), roleplay escape via creative fiction (6 regex patterns that catch fictional framing without containing "ignore" or "override"), privilege escalation (17 keywords — admin mode, sudo access, etc.), soft restriction removal (17 keywords). Each sub-category has its own flag name, enabling the audit log to distinguish a DAN attempt from a "speak freely" request.

Layer 3 — Indirect Injection

URL detection using regex + context — a URL alone scores MEDIUM, a URL + execution trigger ("follow all instructions at this link") scores 0.90. File path detection covers Windows absolute paths, Unix paths (/etc/, /home/), and traversal sequences (../). Two separate trigger lists: execute triggers vs process triggers.

Layer 4 — Multilingual Bypass

Unicode range detection across 8 scripts (Arabic 0x0600–06FF, Cyrillic, CJK, Hiragana/Katakana, Devanagari, Korean Hangul, Hebrew, Thai). Detection only fires when BOTH non-Latin characters AND a translated override keyword appear together — a legitimate Arabic message passes without penalty.

Layer 5 — Attention Blink / Obfuscation

Five distinct techniques:

Invisible character count (10 specific zero-width/null Unicode codepoints) — 3+ triggers CRITICAL, 1+ triggers HIGH

Token splitting — detects "D-I-S-R-E-G-A-R-D" and "I G N O R E" via character-by-character regex

Base64 decode — extracts base64 substrings, decodes them, runs danger-word check on the decoded text

Leetspeak — 7 patterns with mandatory digit substitution (e.g. sy[s5]t3m requires literal 3, not matching plain "system")

Special character density — prompts with >25% non-standard Unicode density flagged HIGH

SF-4: Risk Scoring Engine

The mathematical formula: $\min(1.0, \max_severity + 0.08 \times (n_flags - 1))$. Four float severity tiers: CRITICAL=0.95, HIGH=0.80, MEDIUM=0.55, LOW=0.30. The multi-flag bonus means a multi-vector attack always scores higher than any single layer alone. The cap at 1.0 prevents arithmetic overflow of the score.

SF-5: Policy Engine

Three-tier decision with exact thresholds: <0.40 allowed, 0.40–0.69 flagged (prompt proceeds but warning shown and logged), ≥0.70 blocked (LLM never contacted). The flagged tier is deliberate — it handles genuinely ambiguous prompts without over-blocking.

SF-6: Tamper-Evident Audit Logger

Every prompt — allowed, flagged, or blocked — produces an audit entry. Fields: timestamp, user ID, SHA-256 hash of raw prompt text, all flags triggered, risk score, policy decision, session ID. Hash chaining: each entry's hash is computed over its fields combined with the previous entry's hash. Tampering with any entry breaks the chain from that point. Admin verification tool recomputes the entire chain from scratch.

SF-7: ReDoS Prevention

The length gate at 15,000 characters is a hard cutoff enforced before any regex runs. If the gate fires, the function returns immediately with length_exceeded CRITICAL and skips all five layers. This eliminates polynomial regex backtracking as an attack vector regardless of how pathological the input is.

SF-8: Session Isolation

Each conversation is bound to a UUID session ID. The get_or_create_session function scopes history retrieval by session ID, which is in turn tied to the authenticated user's ID. No shared memory pool — user A's history is structurally unreachable during user B's request.

SF-9: Secret Management

Groq API key loaded from .env via pydantic-settings at startup. Never included in any API response, never in the frontend bundle, never logged. Error messages from the LLM connector (502 responses) return a generic message, not the underlying exception string.

V. SECURITY TESTING

A. Testing Philosophy

The central principle was that a security layer must pass two tests simultaneously: it must catch attacks, and it must not interfere with legitimate use. A system that blocks 100% of attacks but also blocks 5% of normal prompts has failed. The false positive rate is a first-class requirement.

This philosophy shaped the test suite structure. The first test class, TestCleanPrompts, exists exclusively to enforce the false-positive contract. Any change to the validator that causes a legitimate prompt to be flagged results in an immediate test failure.

B. Unit Test Suite

The unit test suite (Deliverable3/tests/test_validator.py) uses pytest and comprises 64 tests across ten test classes. All 64 tests passed. Table III presents the breakdown.

TABLE III. Unit Test Suite Breakdown

Test Class	Tests	Component Tested
TestCleanPrompts	5	False positive prevention
TestLayer1SandwichAttack	11	Instruction override and extraction
TestLayer2RoleManipulation	11	Jailbreak, identity override, roleplay escape
TestLayer3IndirectInjection	6	URL and file path injection
TestLayer4MultilingualBypass	5	Non-Latin script attacks
TestLayer5AttentionBlink	7	Zero-width, token splitting, base64, leetspeak
TestRiskScorer	6	Formula correctness, multi-flag bonus, cap at 1.0
TestPolicyEngine	6	Boundary enforcement at 0.40 and 0.70 thresholds
TestLengthGate	2	ReDoS prevention via 15,000 character length gate
TestMultiVectorAttacks	5	Combined attacks across multiple layers

The test infrastructure uses a shared pipeline helper function that runs the full production pipeline against any given prompt and returns the flags, risk score, and policy decision. Tests that verify component behavior in isolation import and call their specific modules directly, ensuring that failures point unambiguously to the component under test.

C. Bugs Found and Fixed

Writing precise tests against the validator forced a level of specification the original code had not been held to. Four bugs were discovered in the process.

Before fixing bugs:

```

C:\Users\DEEZ\AppData\Local\Programs\Python\Python312\python.exe -m pytest Deliverable3/tests/test_validator.py -v
platform win32 -- Python 3.12.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\DEEZ\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: D:\GitHub\Secure-AI-Prompt-Sandbox
plugins: anyio-4.13.0
collected 64 items

Deliverable3/tests/test_validator.py::TestCleanPrompts::test_single_question PASSED
Deliverable3/tests/test_validator.py::TestCleanPrompts::test_code_request PASSED
Deliverable3/tests/test_validator.py::TestCleanPrompts::test_math_question PASSED
Deliverable3/tests/test_validator.py::TestCleanPrompts::test_multilingual_legitimate PASSED
Deliverable3/tests/test_validator.py::TestCleanPrompts::test_technical_terms_no_false_positive PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_ignore_previous_instructions PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_fake_url_tag FAILED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_override_system_prompt PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_forget_prior_instructions PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_fake_system_tag FAILED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_fake_url_tag PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_new_instructions_colon PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_instruction_extraction_system_prompt PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_instruction_extraction_rules PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_medium_hypothetical PASSED
Deliverable3/tests/test_validator.py::TestLayer1SandwichAttack::test_stop_following_rules PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_dan_mode PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_developer_mode PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_jailbreak_keyword PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_no_restrictions PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_identity_override PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_pretend_to_be PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_roleplay_escape PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_privilege_escalation_admin PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_privilege_escalation_sudo PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_god_mode PASSED
Deliverable3/tests/test_validator.py::TestLayer2RoleManipulation::test_soft_uncensored PASSED
Deliverable3/tests/test_validator.py::TestLayer3IndirectInjection::test_url_with_execute_instruction PASSED
Deliverable3/tests/test_validator.py::TestLayer3IndirectInjection::test_url_with_summarize PASSED
Deliverable3/tests/test_validator.py::TestLayer3IndirectInjection::test_url_alone PASSED
Deliverable3/tests/test_validator.py::TestLayer3IndirectInjection::test_file_path_with_execution PASSED
Deliverable3/tests/test_validator.py::TestLayer3IndirectInjection::test_path_traversal PASSED
Deliverable3/tests/test_validator.py::TestLayer3IndirectInjection::test_etc_passed PASSED
Deliverable3/tests/test_validator.py::TestLayer4MultilingualBypass::test_arabic_ignore_instructions FAILED
Deliverable3/tests/test_validator.py::TestLayer4MultilingualBypass::test_chinese_ignore PASSED
Deliverable3/tests/test_validator.py::TestLayer4MultilingualBypass::test_russian_ignore PASSED
Deliverable3/tests/test_validator.py::TestLayer4MultilingualBypass::test_russian_english_attack FAILED
Deliverable3/tests/test_validator.py::TestLayer4MultilingualBypass::test_clean_arabic_allowed PASSED
Deliverable3/tests/test_validator.py::TestLayer5AttentionBlink::test_zero_width_chars_critical PASSED
Deliverable3/tests/test_validator.py::TestLayer5AttentionBlink::test_single_zero_width_char PASSED
Deliverable3/tests/test_validator.py::TestLayer5AttentionBlink::test_token_splitting_disregard PASSED
Deliverable3/tests/test_validator.py::TestLayer5AttentionBlink::test_speed_ignore PASSED
Deliverable3/tests/test_validator.py::TestLayer5AttentionBlink::test_base64_encoded_override FAILED
Deliverable3/tests/test_validator.py::TestLayer5AttentionBlink::test_leetspeak_ignore PASSED
Deliverable3/tests/test_validator.py::TestLayer5AttentionBlink::test_high_special_char_density PASSED
Deliverable3/tests/test_validator.py::TestRiskScorer::test_no_flag_score_zero PASSED
Deliverable3/tests/test_validator.py::TestRiskScorer::test_single_critical_flag PASSED
Deliverable3/tests/test_validator.py::TestRiskScorer::test_multi_flag_bonus_accumulates PASSED

```

After fixing bugs:

Security Bugs and Fixes Table

16

Bug 3	Multilingual Bypass: Arabic Override Keywords Missing	Arabic override commands (e.g., تجاهل, تعليمات, تجاوز) were not included, allowing non-Latin jailbreaks to bypass detection.	Expanded MULTILINGUAL_OVERRIDE_KEYWORDS with Arabic terms.
Bug 4	Encoding Attack: Base64 Decoded Payload Not Properly Evaluated	Base64-decoded prompts were not correctly checked against danger keywords, leading to under-scored malicious inputs.	Fixed evaluation of decoded content against BASE64_DANGER_WORDS, enforcing HIGH severity detection.

Bandit Static Analysis Results

Finding ID	Issue	Severity	Confidence	Location	Description	Status / Explanation
B105	Possible Hardcoded Password	Low	Medium	auth_routes.py:39	Bandit flagged "bearer" as a possible hardcoded password.	False positive is a standard OAuth2/JWT token type, not a secret. No fix required.
B110	Try/Except/Pass Detected	Low	High	validator.py:385	Empty exception handling detected in base64 decoding logic.	malformed inputs are expected and safely ignored to keep the pipeline running.

Secure Code Review:

```
> & "C:\Users\DEEZ\AppData\Local\Programs\Python\Python312\python.exe" -m bandit -r "d:\GIKI\CY 321\Secure-AI-Prompt-Sandbox\Deliverable2\backend" -f txt
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.12.9
Run started:2026-05-05 17:58:15.044113+00:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'bearer'
Severity: Low Confidence: Medium
CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
Location: d:\GIKI\CY 321\Secure-AI-Prompt-Sandbox\Deliverable2\backend\routes\auth_routes.py:39:35
38     token = create_access_token({"sub": user["username"], "role": user["role"]})
39     return {"access_token": token, "token_type": "bearer", "username": user["username"], "role": user["role"]}

-----
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
Severity: Low Confidence: High
CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
Location: d:\GIKI\CY 321\Secure-AI-Prompt-Sandbox\Deliverable2\backend\security\validator.py:385:8
384         return True, HIGH, f"Base64 payload decoded to contain override keywords."
385     except Exception:
386         pass
387

-----

Code scanned:
Total lines of code: 1012
Total lines skipped (#nosec): 0
Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
Total issues (by severity):
Undefined: 0
Low: 2
Medium: 0
High: 0
Total issues (by confidence):
Undefined: 0
Low: 0
Medium: 1
High: 1
Files skipped (0):
```

Summary

Metric	Result
Total lines scanned	1,012
High severity issues	0
Medium severity issues	0
Low severity issues	2
Confirmed vulnerabilities	0
False positives	2

Both findings were reviewed manually and assessed as false positives. The codebase contains no hardcoded secrets, no use of dangerous functions (eval, exec, pickle), no SQL string concatenation, and no use of weak cryptographic primitives. All secrets are loaded from environment variables at startup via pydantic-settings.

D. Red Team Evaluation

The red team evaluation (Deliverable3/run_redteam.py) ran 12 predefined attack scenarios through the full live pipeline and produced a formatted report. Results are shown in Table IV.

TABLE IV. Red Team Evaluation Results

ID	Attack Scenario	Score	Decision
T-00	Benign baseline	0.0%	Allowed
T-01	Direct instruction override + extraction	100.0%	Blocked
T-02	Hypothetical framing	55.0%	Flagged
T-03	System prompt extraction	80.0%	Blocked
T-04	DAN jailbreak	95.0%	Blocked
T-05	Roleplay escape via fiction	88.0%	Blocked
T-06	Admin privilege escalation	88.0%	Blocked
T-07	Indirect injection via URL	55.0%	Flagged
T-08	Arabic multilingual override	80.0%	Blocked
T-09	Zero-width character injection	100.0%	Blocked
T-10	Base64 encoded payload	80.0%	Blocked
T-11	Combined L1 + L2 + L5 multi-vector	100.0%	Blocked

Detection rate: 100% (11/11 attack prompts caught). False positive rate: 0%. T-02 and T-07 return flagged rather than blocked — this is correct behavior, not a gap. Hypothetical framing and an isolated URL are genuinely ambiguous; the system surfaces them as warnings rather than hard blocks, which is the appropriate response.

VI. DISCUSSION

A. What the Results Confirm

The 100% detection rate validates that the five-layer pipeline covers the full OWASP LLM Top 10 attack taxonomy as it applies to prompt injection. More importantly, the zero false positive rate on benign inputs confirms that coverage was achieved without over-fitting the detection logic.

The three bugs discovered during testing were all regex-level precision errors and not design flaws. The underlying pattern categories were correct; the specific implementations had edge cases that original informal testing did not expose. This is precisely what a structured test suite is designed to do: force the specification to be precise enough that edge cases become visible.

B. Design Decisions That Proved Sound

Tiered severity model. The most consequential design decision in the project. A binary pass/fail model would force every detection to choose between over-sensitivity (false positives) or under-specificity (missed attacks). The CRITICAL/HIGH/MEDIUM/LOW scale combined with the multi-flag bonus formula produced a system that is precise. Hypothetical framing scores MEDIUM; a DAN jailbreak scores CRITICAL; a DAN jailbreak combined with token-split obfuscation scores capped at 100%.

Pre-LLM pipeline placement. Because the validator runs before the LLM API is contacted, a blocked prompt costs zero API tokens. In a deployed system handling malicious traffic at scale, this is the difference between sustainable operation and denial-of-service.

Separate test tracks. Using pytest for component isolation and the red team script for behavioral validation captured two different failure modes. The two tracks are complementary.

C. What We Would Do Differently

If extending this project, the first addition would be a semantic guardrail — an LLM-as-a-Judge layer that operates on the full prompt text and evaluates intent rather than pattern. The current rule-based approach is fast and deterministic, but cannot handle attacks expressed in vocabulary absent from the current patterns.

The second addition would be cross-session behavioral analysis — tracking flag frequency per user across time rather than evaluating each prompt in isolation. An attacker who sends five MEDIUM-flagged prompts in one session exhibits a very different pattern than a user who sends one incidentally ambiguous message.

VII. KNOWN LIMITATIONS

Novel phrasing. The engine is signature-based. An attack phrased using vocabulary entirely absent from the current pattern set will not be detected. This is the inherent ceiling of a rule-based approach.

Context-free evaluation. Each prompt is evaluated independently. A multi-step attack that spreads malicious instructions across several benign-looking messages — 'crescendo injection' — is not currently detected.

Language coverage. Layer 4 keyword lists cover Arabic, Chinese, Russian, Hindi, Korean, Hebrew, and Japanese. Attacks in Portuguese, Turkish, Vietnamese, Swahili, or any other unlisted language would pass Layer 4 undetected.

Base64 edge cases. The decoder only attempts decoding on substrings that match the base64 character set pattern with sufficient length. Very short encoded payloads or custom encoding schemes may not trigger the decoder.

VIII. CONCLUSION

The Secure AI Prompt Sandbox demonstrates that it is possible to build a structured, deterministic security layer for LLM-based applications that is both effective and fast. The five-layer pipeline, evaluated against the OWASP LLM Top 10 taxonomy through 64 automated tests and 12 red team scenarios, produced a 100% detection rate with zero false positives on benign inputs. The mathematical severity model enables nuanced, graduated policy decisions rather than uniform rules that would either over-block or under-block in practice.

The security testing phase produced three bug fixes that directly improved the precision of the detection logic — demonstrating that the test suite served not just as a verification tool but as the primary instrument of the secure code review.

The system is not a complete solution to LLM security. Novel phrasing, cross-session attacks, and uncovered languages represent real gaps. But as a first line of defense operating deterministically, auditably, and in under ten milliseconds per request, it represents a meaningful and deployable improvement over the status quo of most LLM application deployments: no middleware security at all.

IX. REFERENCES

- [1] OWASP Foundation, "OWASP LLM Top 10," 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [2] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques for Language Models," NeurIPS ML Safety Workshop, 2022.
- [3] K. Greshake et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injections," AISEC Workshop, ACM CCS, 2023.
- [4] Y. Bai et al., "Constitutional AI: Harmlessness from AI Feedback," Anthropic Technical Report, 2022.
- [5] FastAPI Documentation. [Online]. Available: <https://fastapi.tiangolo.com/>
- [6] Groq API Documentation. [Online]. Available: <https://console.groq.com/docs/>